RAG Caching System

From Theory to Implementation

3-Tier Cache · SQLite & Redis Backends · FastAPI Integration

Agenda

PART 1

Caching Theory

What, why, how — core concepts, embeddings & invalidation

PART 2

Architecture Design

3-tier cache flow, tradeoffs, performance & cost analysis

PART 3

Code Deep Dive

File-by-file walkthrough — patterns, schema & endpoints

PART 4

Infrastructure

Redis intro, Docker, docker-compose & RedisInsight

PART 5

Setup & Beyond

Step-by-step guide, testing checklist & future roadmap

03

Caching Theory

What is caching, why it matters, and how it works

What is Caching?

Store results of expensive operations for fast reuse

WITHOUT Cache

```
User asks question
  → Pinecone retrieval    ~500ms
  → Reranking             ~300ms
  → LLM generation        ~3000ms
  Total: ~4 seconds, ~$0.001

Same question 10s later?
  → Same pipeline again!
  → Same cost, same wait
```

WITH Cache

```
First ask (cache miss):
  → Full pipeline   ~4 seconds
  → Store result in cache

Second ask (cache hit):
  → Check cache     ~5ms
  → Return instantly!

  99% faster · 100% cost saved
```

Why Caching in RAG?

Latency

3-8 seconds per question is poor UX. Cache delivers <100ms responses.

Cost

LLM calls cost ~$0.001 each. At 10K queries/day = $300/month wasted on repeats.

Redundancy

Users ask the same thing in different ways. ~60% of queries are near-duplicates.

Scale

Cache absorbs traffic spikes. 10x more users, same API costs.

Core Cache Concepts

Cache Hit

Query found in cache.
Return stored result instantly.
No expensive computation.

Cache Miss

Query NOT in cache.
Run full pipeline.
Store result for next time.

TTL (Time-to-Live)

Auto-expiry timer.
Entries die after N seconds.
Prevents serving stale data.

KEY INSIGHT

A cache entry must pass existence + TTL + version checks to be a valid hit

Cache Invalidation — The Hard Problem

"There are only two hard things in CS: cache invalidation and naming things."

TTL-Based

Entries auto-expire after a set time (7 days exact/semantic, 1 day retrieval). Simple and automatic.

Version Tagging

Each entry stores a doc_version. When new docs are ingested, version bumps and stale entries are skipped.

Manual Clear

POST /cache/clear endpoint wipes all tiers. Full control when you need it.

Document Version — How It Works

Global counter prevents stale answers after document uploads

1

User asks "Apple revenue?"

```
Cache miss → full pipeline
Answer stored with
doc_version = 3
```

→

2

Same question asked again

```
Tier 1 exact hit!
Version check: entry=3,
current=3 → VALID
```

→

3

Admin uploads new earnings PDF

```
bump_doc_version()
counter: 3 → 4
All old entries stale
```

→

4

Same question asked again

```
Entry found but
entry=3, current=4
→ STALE → full pipeline
```

IMPORTANT: This is a global invalidation — ALL cached entries become stale, not just entries related to the uploaded doc. Per-document invalidation is a future improvement.

Embeddings & Cosine Similarity

How Tier 2 & 3 determine if two questions are semantically similar

What is an Embedding?

```
Text → fixed-size number array (vector)
that captures meaning in math space.

"Apple revenue?"
 → [0.023, -0.015, 0.089, ..., -0.041]
    (1536 numbers)

"Apple earnings?"
 → [0.022, -0.016, 0.088, ..., -0.040]
    (very similar numbers!)

"How is the weather?"
 → [0.078, 0.034, -0.023, ..., 0.056]
    (completely different!)
```

Cosine Similarity

```
Formula:
  cos(A,B) = A·B / (||A|| × ||B||)

Result range:
  1.0  = identical meaning
  0.0  = completely unrelated
 -1.0  = opposite (rare in practice)

Our thresholds:
  Tier 2: ≥ 0.95 (strict, rephrasings)
  Tier 3: ≥ 0.90 (loose, same context)

Implementation: numpy dot product
  ~1ms for 1000 comparisons
```

08

Architecture Design

3-tier cache with swappable backends

3-Tier Cache Architecture

Each tier catches a different kind of redundancy before the expensive pipeline runs

User Question → Tier 1 Exact → (miss →) Tier 2 Semantic → (miss →) Tier 3 Retrieval → (miss →) Full RAG Pipeline

Tier 1: Exact Cache

Catches

Exact duplicate questions

Method

hash(normalize(query)) → O(1)

Saves

LLM + Pinecone + Reranker

Speed

1-10ms · Cost: $0.00

Tier 2: Semantic Cache

Catches

Rephrasings, typos, synonyms

Method

embed → cosine sim ≥ 0.95

Saves

LLM + Pinecone + Reranker

Speed

50-200ms · Cost: ~$0.00002

Tier 3: Retrieval Cache

Catches

Same context, different question

Method

embed → cosine sim ≥ 0.90

Saves

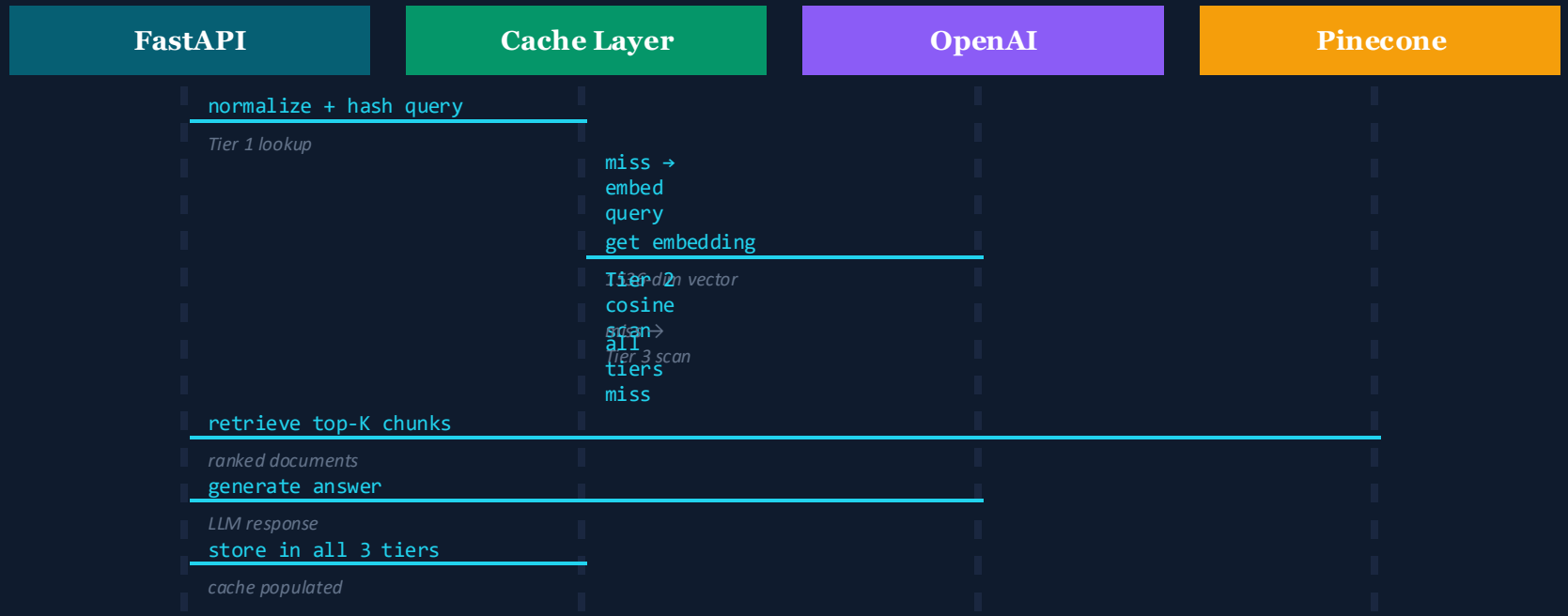
Pinecone + Reranker only

Speed

2-4s (still calls LLM)

Full Request Lifecycle

What happens when a user sends a question to /chat



Total time (cache miss): ~5 seconds · Next identical query (cache hit): ~5ms · Rephrasing: ~100ms

Tier 1 — Exact Cache

1. Normalize

`"What was Apple's Revenue??" → "what was apple's revenue"`

2. Hash (SHA256)

`"what was apple's revenue" → "0d9515fdd008..."`

3. Lookup

`hash → database[hash] → O(1) instant lookup`

4. Validate

`Check TTL expiry + document version tag`

5. Return

`Cached answer + sources returned to user`

RESPONSE TIME: 1-10ms

COST: $0.00 (no API calls)

Tier 2 — Semantic Cache

Catches rephrasings using vector embeddings + cosine similarity

COSINE SIMILARITY EXAMPLES

"Apple revenue?"	≈ "What was Apple's revenue?"	0.97	HIT
"AAPL earnings"	≈ "Apple quarterly earnings"	0.95	HIT
"Nike sales"	≈ "Apple revenue"	0.34	MISS

```
Threshold: 0.95 (configurable)
Model: text-embedding-3-small
Dimensions: 1536
Cost per lookup: ~$0.00002
```

```
How it works:
1. Embed incoming query via OpenAI
2. Compare against ALL cached embeddings
3. Best match above threshold = HIT
```

Tier 3 — Retrieval Cache

Caches the retrieved chunks, not the answer — still calls LLM for a fresh response

What Tier 3 Caches

```
Question embedding → Chunks from Pinecone

Different questions that need the
same document context:
  Q1: "Apple Q2 revenue?"
  Q2: "Apple Q2 expenses?"
  → Same chunks, different answers
```

What Tier 3 Saves

```
Skips:  Pinecone retrieval  (~500ms)
        Reranking           (~300ms)

Still calls: LLM generation  (~2-4s)

Lower threshold: 0.90
(matching context, not question)
```

CACHE PROMOTION

When Tier 3 hits, we generate an answer with LLM + cached chunks. That new answer is promoted into Tier 1 + Tier 2. Next time the same or similar question appears, it becomes a faster Tier 1 or Tier 2 hit. The cache gets smarter over time.

Cache Promotion — Self-Learning Cache

Tier 3 hits automatically create faster Tier 1 + Tier 2 entries for next time

Visit 1 — "Apple Q2 revenue?" → All Cache Miss

Tier 1 MISS → Tier 2 MISS → Tier 3 MISS → Full RAG Pipeline → Store in ALL 3 Tiers

Visit 2 — "What was Apple's Q2 revenue?" → Tier 2 Semantic Hit!

Tier 1 MISS (different text) → Tier 2 HIT! ✓ (cosine 0.97) → Return Instantly (~100ms)

99.8% faster

Each visit makes the cache smarter — Tier 3 promotes to Tier 1+2, Tier 2 catches rephrasings of promoted entries

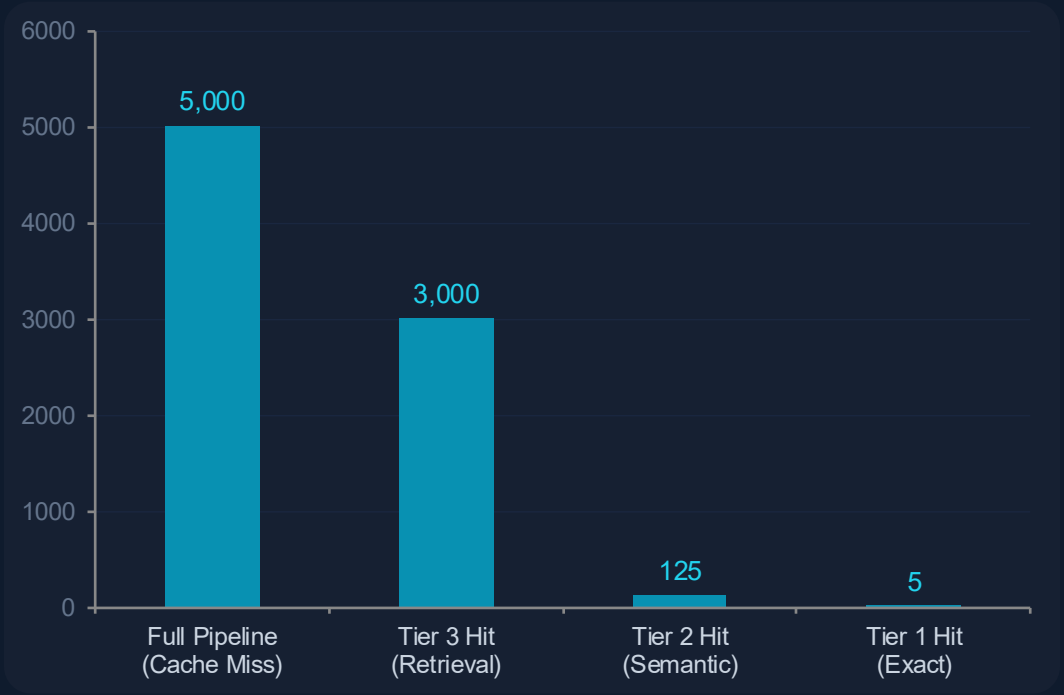
Backend Toggle — SQLite vs Redis

Same interface, different storage. Swap with one env var.

Aspect	SQLite	Redis
Storage	File on disk	In-memory + disk
Speed (read)	~0.5ms	~0.1ms
TTL handling	Manual in Python	Native auto-expiry
Concurrency	WAL mode (~10)	Thousands of clients
Infrastructure	None (just a file)	Docker / Cloud service

```
# .env — one line switches everything
CACHE_BACKEND=sqlite   →  SQLite file (zero infra)
CACHE_BACKEND=redis    →  Redis via Docker (production)
```

Performance Impact



5000ms
Full pipeline

3000ms
Tier 3 (skip retrieval)

125ms
Tier 2 (semantic hit)

5ms
Tier 1 (exact hit)

Tier 1 exact hit = 1000x faster than full pipeline

Cost Analysis

Per Query — No Cache

```
Pinecone retrieval:    $0.0001
Reranking:             $0.0001
LLM generation:        $0.001
──────────────────────
Total per question:    $0.0012
```

Per Query — Cache Hit

```
Tier 1 (exact):        $0.00
Tier 2 (semantic):     $0.00002
Tier 3 (retrieval):    $0.001
──────────────────────
98% cost reduction on Tier 2 hit
```

MONTHLY PROJECTION — 10,000 queries/day, 60% cache hit rate

$360
/month without cache

→

$148
/month with cache

SAVE $212/mo

Code Deep Dive

File-by-file walkthrough of the caching system

Project File Structure

```
apps/cache/
├── __init__.py      ← Factory
├── base.py          ← Abstract Interface
├── embeddings.py    ← Math/Utils
├── sqlite_backend.py ← SQLite
└── redis_backend.py  ← Redis

apps/config.py      ← All settings
apps/api.py         ← FastAPI endpoints
docker-compose.yml ← Redis container
.env                ← Env variables
```

config.py

All cache settings — TTLs, thresholds, backend choice, DB paths

base.py

Abstract class — 11 methods that every backend must implement

embeddings.py

Shared utils — embed, normalize, hash, cosine similarity

sqlite / redis

Two implementations of the same interface — swap with one env var

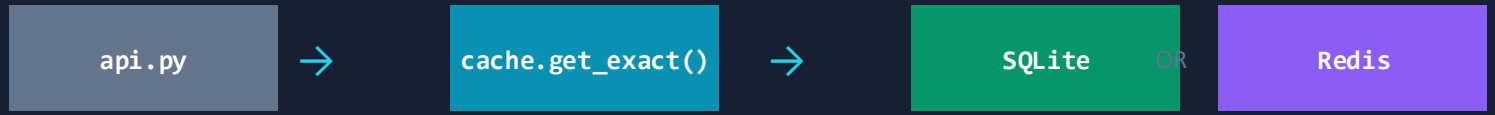
api.py

FastAPI endpoints — /chat with cache, /cache/stats, /cache/clear

Design Patterns Used

Pattern	Where	Why
Strategy	CacheBackend ABC + SQLite/Redis	Swap storage without changing any other code
Factory	get_cache_backend()	Centralized creation — one place decides which backend
Singleton	_backend_instance	Don't create new DB connections per request
Lazy Init	_embeddings_client	Don't call OpenAI until we actually need an embedding
Template Method	[illegible]	Same validation logic, different

STRATEGY PATTERN IN ACTION



config.py — Configuration

```
CACHE_BACKEND   = "sqlite"  # or "redis"
CACHE_ENABLED   = true      # master on/off

# Tier 1 & 2: 7 day TTL
EXACT_CACHE_TTL    = 604800    # seconds
SEMANTIC_CACHE_TTL = 604800

# Tier 2: cosine similarity threshold
SEMANTIC_CACHE_THRESHOLD = 0.95

# Tier 3: 1 day TTL, lower threshold
RETRIEVAL_CACHE_TTL       = 86400
RETRIEVAL_CACHE_THRESHOLD = 0.90

# Storage paths
DATABASE_PATH   = "data/rag_cache.db"
REDIS_URL       = "redis://localhost:6379/0"
OPENAI_EMBED_MODEL = "text-embedding-3-small"
```

base.py — The Abstract Interface

Defines WHAT the cache must do, not HOW — any new backend just implements this

TIER 1 (Exact)

```
get_exact(query_hash)
set_exact(hash, q, answer, sources)
```

TIER 2 (Semantic)

```
get_semantic(embedding, threshold)
set_semantic(q, embedding, answer, sources)
```

TIER 3 (Retrieval)

```
get_retrieval(embedding, threshold)
set_retrieval(q, embedding, chunks)
```

MANAGEMENT

```
get_doc_version() / bump_doc_version()
clear_all() / get_stats()
cleanup_expired()
```

embeddings.py — Shared Utilities

`embed_query(text)`	→ 1536-dim vector via OpenAI ($0.00002/call)
`normalize_query(text)`	→ lowercase, strip, collapse spaces, trim punctuation
`hash_query(text)`	→ SHA256 hex digest for Tier 1 keys
`cosine_similarity(a, b)`	→ dot(A,B) / (||A|| × ||B||) → float [-1, 1]
`embedding_to_bytes()`	→ numpy float32 → compact binary (6KB vs 50KB)
`bytes_to_embedding()`	→ binary → numpy float32 array

```
Cosine = 1.0 (identical meaning)    0.0 (unrelated)    -1.0 (opposite meaning)
```

SQLite Schema — 4 Tables

cache_metadata	exact_cache	semantic_cache	retrieval_cache
key TEXT PK value TEXT	query_hash TEXT PK question_text TEXT answer_text TEXT sources_json TEXT doc_version INT created_at REAL ttl_seconds INT hit_count INT	id INTEGER PK question_embedding BLOB question_text TEXT answer_text TEXT sources_json TEXT doc_version, ttl_seconds hit_count, last_hit_at	id INTEGER PK question_embedding BLOB chunks_json TEXT doc_version INT created_at REAL ttl_seconds INT hit_count INT
Stores doc_version	*Tier 1 O(1) lookup*	*Tier 2 BLOB=6KB*	*Tier 3 Chunks*

WAL Mode · BLOBs for embeddings · Brute-force cosine scan · Thread-safe connections

sqlite_backend.py — SQLite Implementation

WAL Mode
Write-Ahead Logging — concurrent reads while writing. Critical for web servers.

4 Tables
cache_metadata, exact_cache, semantic_cache, retrieval_cache

BLOBs
Embeddings stored as numpy float32 bytes (6KB per 1536-dim vector)

Brute-Force Scan
Tier 2/3: loads ALL entries, computes cosine similarity in Python

Thread-Safe
New connection per operation — no shared state across requests

```
get_exact() flow:  Lookup by hash → Check TTL → Check doc_version → Update hit stats → Return
```

redis_backend.py — Redis Implementation

REDIS KEY STRUCTURE

```
cache:exact:<sha256>        STRING
cache:semantic:<uuid>       STRING
cache:retrieval:<uuid>      STRING
cache:metadata:doc_version  STRING
cache:semantic:index        SET
cache:retrieval:index       SET
```

REDIS ADVANTAGES

```
Native TTL via SETEX
  → Auto-expiry, no cleanup code

Atomic INCR
  → doc_version, no race conditions

Sub-millisecond reads
  → Data lives in RAM

SET indexes
  → Track all entry IDs for scanning
```

TTL RACE CONDITION

When a key expires via Redis TTL, the SET index still contains the UUID.
get_semantic() detects this: GET returns nil → SREM to clean up the orphaned index entry.
This is the "index cleanup" pattern — Redis handles TTL, we handle index consistency.

api.py — /chat Endpoint Flow

1

Normalize + Hash query

Try Tier 1 exact lookup

→ Hit? Return instantly (1-10ms)

2

Embed query via OpenAI

Try Tier 2 semantic lookup

→ Hit? Return cached answer (50-200ms)

3

Check Tier 3 retrieval

Call LLM with cached chunks

→ Hit? Promote to Tier 1+2 (2-4s)

4

Full RAG pipeline

Store result in ALL 3 tiers

→ Miss: Pinecone + Reranker + LLM (3-8s)

New: GET /cache/stats · POST /cache/clear

Updated API Response

Three new fields added to POST /chat (backward compatible)

```
{
  "answer": "Apple reported...",
  "sources": [...],

  "cache_hit": true,
  "cache_tier": "semantic",
  "response_time_ms": 42.17
}
```

cache_hit

bool — was this cached?

cache_tier

"exact" | "semantic" | "retrieval" | null

response_time_ms

Total request time in milliseconds

GET /cache/stats Response

```
{ "backend": "sqlite", "doc_version": 3,
  "exact": {"entries": 42, "total_hits": 156},
  "semantic": {"entries": 38, "total_hits": 67},
  "retrieval": {"entries": 15, "total_hits": 12} }
```

26

Setup & Demo

Docker, Redis, switching backends, and testing

What is Redis?

RE-mote DI-ctionary S-erver — an in-memory key-value store

In-Memory Storage

RAM is ~100,000x faster than disk.
All data lives in memory.

Simple Protocol

RESP wire protocol — minimal overhead. No SQL parsing.

Single-Threaded

No locks, no context switching.
One thread = zero contention.

Optimized C Structs

Hash tables, skip lists, zip lists tuned for in-memory access.

100,000+

operations/sec on a single server

Used by: Twitter, GitHub, Snapchat, Stack Overflow, Airbnb

docker-compose.yml — Container Setup

rag-redis · Redis 7 Alpine

```
image: redis:7-alpine  (~30MB)
port:  6379 → 6379
volume: redis_data:/data
restart: unless-stopped
healthcheck: redis-cli ping

Alpine Linux = ~5MB base
(vs redis:7 at ~130MB)
```

rag-redisinsight · Web UI

```
image: redis/redisinsight:latest
port:  5540 → 5540
depends_on: redis
restart: unless-stopped

Browse keys, TTLs, memory usage
Connect: host="redis" port=6379
(service name, not localhost)
```

```
docker compose up -d       # start both containers (background)
docker compose down         # stop (data preserved in volume)
docker compose down -v      # stop + DELETE all data
docker compose logs redis   # view Redis logs
```

Setup Guide

SQLite (Default — Zero Setup)

```
# 1. Set .env (already default)
CACHE_BACKEND=sqlite
CACHE_ENABLED=true

# 2. Start backend
uv run python main.py

# 3. That's it!
# SQLite file auto-created at
# data/rag_cache.db

# 4. Test
curl localhost:8000/cache/stats
```

Redis (Docker Required)

```
# 1. Start Docker Desktop
# Wait for engine ready

# 2. Start Redis container
docker compose up -d

# 3. Verify
docker exec rag-redis redis-cli ping
# → PONG

# 4. Switch backend in .env
CACHE_BACKEND=redis

# 5. Restart: uv run python main.py
```

Testing & Verification

Same question twice	`cache_tier: "exact", time drops ~5000 → ~5ms`
Rephrased question	`cache_tier: "semantic", time ~100ms`
GET /cache/stats	`Entry counts increase after questions`
POST /cache/clear	`All tiers reset to 0`
Upload new document	`doc_version bumps, old cache stale`
Switch sqlite ↔ redis	`Same behavior, different backend`

```
curl -X POST localhost:8000/chat -H "Content-Type: application/json" -d '{"question": "What was Apple revenue?"}'
```

Scaling Considerations

Current: Brute-Force Scan

```
Tier 2/3 loads ALL embeddings,
computes cosine similarity in Python.

  1,000 entries  →    ~1ms (fine)
 10,000 entries  →   ~10ms (fine)
100,000 entries  →  ~100ms (slow!)
```

At Scale: ANN Algorithms

```
Approximate Nearest Neighbor:

  FAISS        — Facebook, local
  HNSW graphs  — sub-ms at 1M+
  RediSearch   — native in Redis
  Qdrant       — managed vector DB
```

REDIS IN PRODUCTION

Managed services: Redis Cloud (30MB free) · AWS ElastiCache · Azure Cache · Upstash

Switch: just change REDIS_URL in .env — zero code changes needed.

Redis Cluster: split keys across nodes → 96GB RAM, ~300K ops/sec
Redis Sentinel: auto-failover if primary dies → promotes replica to primary

Future Improvements

Per-Document Invalidation

Only invalidate entries that used chunks from the changed doc. Currently we invalidate all.

LRU Eviction

Remove least-recently-used entries when cache hits a size limit. Prevent unbounded growth.

Semantic Re-validation

Re-check if cached answers are still correct given newly ingested documents.

Smart Query Router

ML model predicts whether to use cache or skip to pipeline based on query type.

Observability Dashboard

Grafana dashboard with hit rates, latency percentiles, and cost savings in real-time.

Summary & Q&A

- 3-tier cache (Exact → Semantic → Retrieval) catches different redundancy levels
- 1000x speedup on exact hits (5ms vs 5000ms) with 98% cost reduction
- Swappable backends (SQLite for dev, Redis for prod) via Strategy Pattern
- TTL + Version Tags + Manual Clear = robust cache invalidation
- Cache gets smarter over time via Tier 3 → Tier 1+2 promotion

Questions?

RAG Architect — Datasense